SQL
S5

EDC- 2026
Sh. Dowlatshahi

Session 5 :Joins

- Joins: Inner, Outer, Self, Cross

Course content

S1: Introduction to Databases & SQL

S2: DDL

S3: DML

S4: Aggregation and Basic Analysis

S5: Working with Multiple Tables

S6: Advanced Queries

S7: Final presentation & Exam

Import Query

```
COPY table_name (
column_1,
column_2,
column_3,
...
column_n
)
FROM 'file_path/file_name.csv'
DELIMITER ','
CSV HEADER
ENCODING 'UTF8';
```

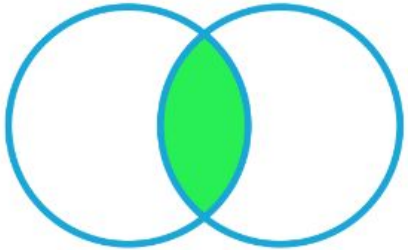
Combine Multiple tables => join

A JOIN in SQL is used to combine rows from two or more tables based on a **related/ matching column** between them.

This is helpful when you want to retrieve data from multiple tables that are related to each other.

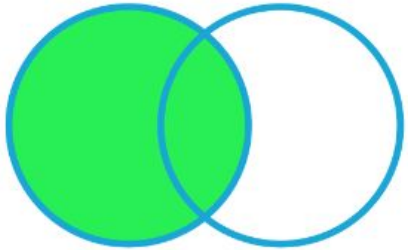
Merging data: types of joins

Inner Join



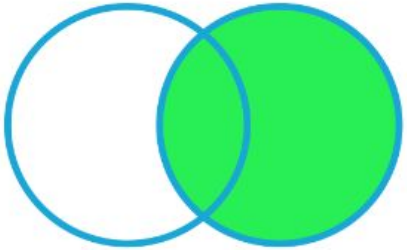
Returns matched rows only

Left Join



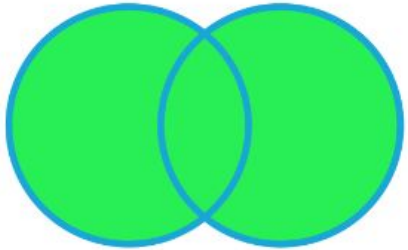
Returns matched rows and left table's rows

Right Join



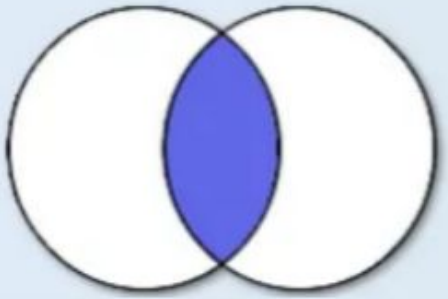
Returns matched rows and right table's rows

Full Join

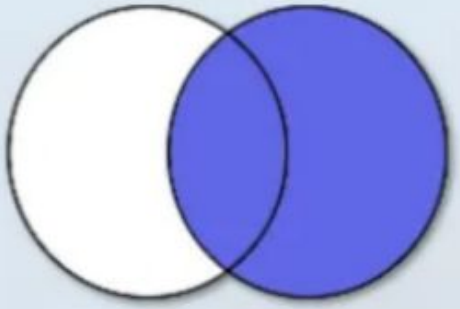


Returns matched rows and both tables' rows

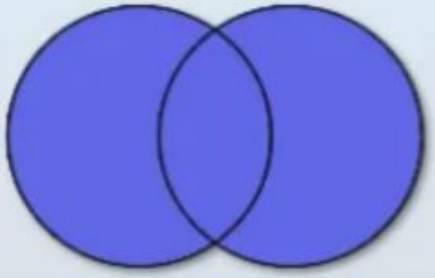
Inner Join



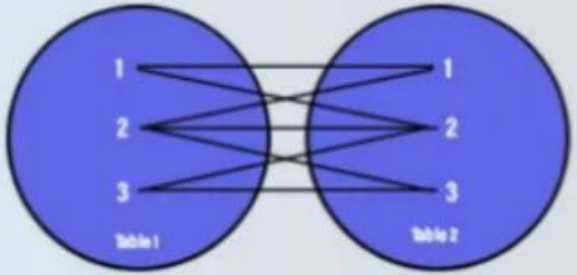
Right Join



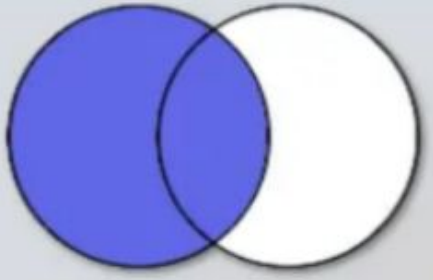
Full Join



Cross Join



Left Join



```
SELECT columns
FROM table1
JOIN table2 ON table1.common_column = table2.common_column;
```

It is an INNER JOIN

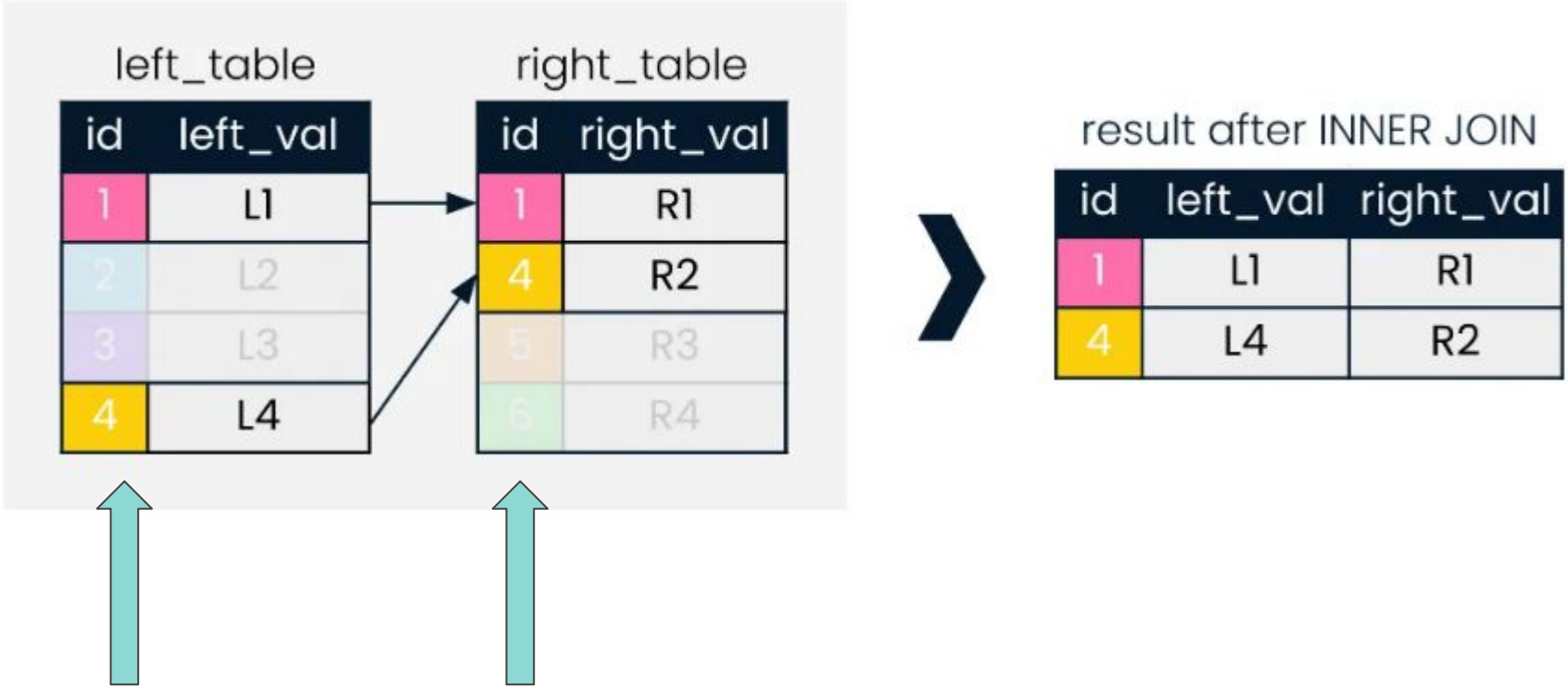
1. INNER JOIN

Returns only the rows where there is a match in both tables.

Use this when you want to see data that exists in both tables.

Example:

Show customers who have placed orders. (only matched customer and order rows are returned.)


left_table
id
left_val
1
L1
2
L2
3
L3
4
L4
right_table
id
right_val
1
R1
4
R2
5
R3
6
R4
result after INNER JOIN
id
left_val
right_val
1
L1
R1
4
L4
R2

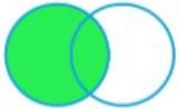
2. LEFT JOIN (Left Outer Join)

Returns all rows from the left table, and matching rows from the right table.

If there is no match, NULLs are shown for the right table.

Use when: You want to keep everything from the left table, even if there's no match in the right.

Merging data: example of a left join



Sales Table		
Customer_id	Value	**Product_id**
1	$80	**123**
2	$5	**456**
3	$500	**789**

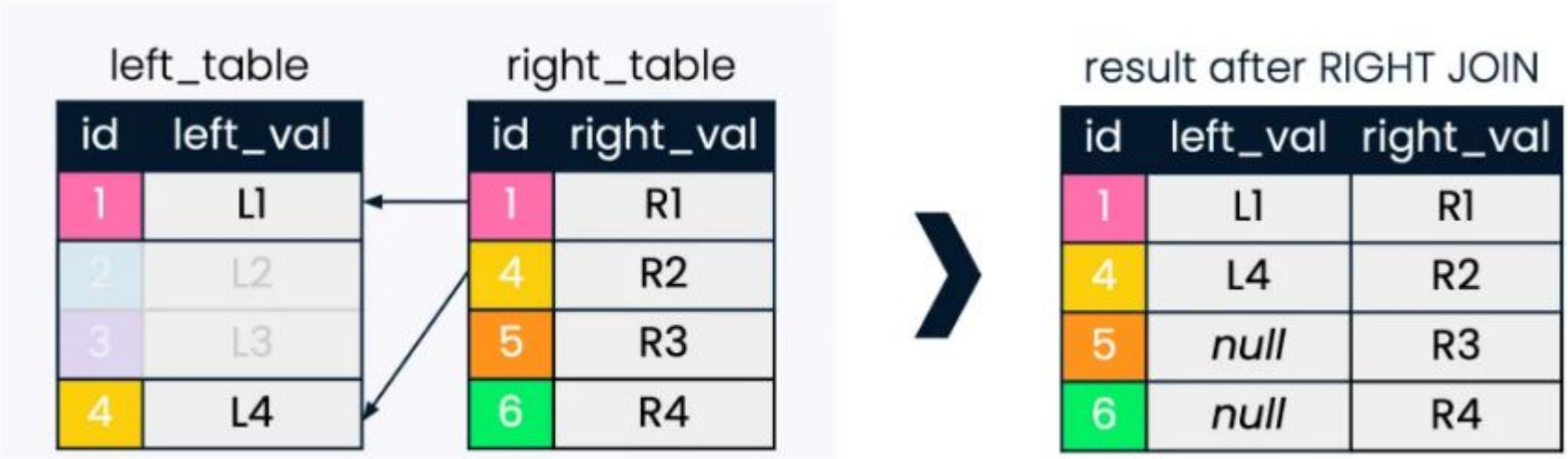
Product Table		
Product_id	Category	Product
123	Shoes	Running Shoes
456	Clothing	Socks

Customer_id	Value	**Product_id**	Category	Product
1	$80	**123**	Shoes	Running Shoes
2	$5	**456**	Clothing	Socks
3	$500	**789**	*null*	*null*

3. RIGHT JOIN (Right Outer Join)

Returns all rows from the right table, and matching rows from the left table. If there’s no match, NULLs are shown for the left table.

Use when: You want to keep everything from the right table, even if there's no match in the left.


left_table
id
left_val
1
L1
2
L2
3
L3
4
L4
right_table
id
right_val
1
R1
4
R2
5
R3
6
R4
result after RIGHT JOIN
id
left_val
right_val
1
L1
R1
4
L4
R2
5
null
R3
6
null
R4

When to Use LEFT vs RIGHT JOIN?

Use LEFT JOIN if your main interest is in the first (left) table.

Use RIGHT JOIN if your main interest is in the second (right) table.

4. FULL JOIN / FULL OUTER JOIN

Returns all rows from both tables. If a row doesn’t have a match in the other table, NULLs fill in the missing side.

Use when: You want everything, matched and unmatched rows from both tables.

left_table

id	left_val
1	L1
2	L2
3	L3
4	L4

right_table

id	right_val
1	R1
4	R2
5	R3
6	R4



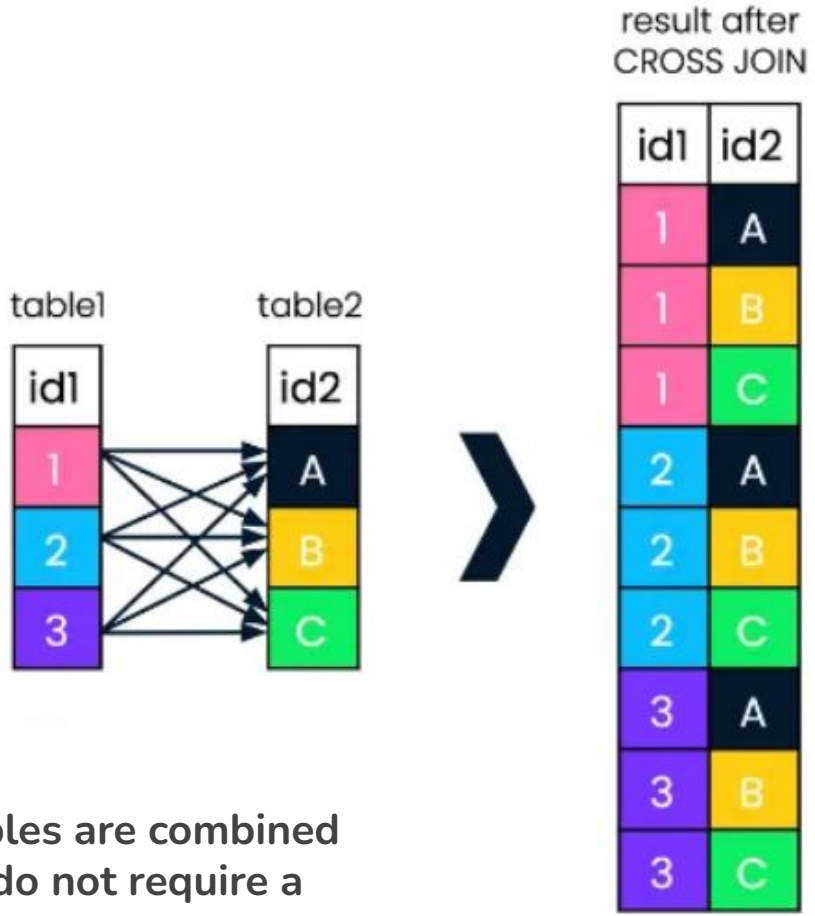
result after FULL JOIN

id	left_val	right_val
1	L1	R1
2	L2	*null*
3	L3	*null*
4	L4	R2
5	*null*	R3
6	*null*	R4

5. CROSS JOIN

Returns all possible combinations of rows from the two tables. If one table has 3 rows and the other has 4, the result will have 12 rows.

Use when: You need a Cartesian product: every row from table A combined with every row from table B.



In a CROSS JOIN, the tables are combined without a condition and do not require a common column.

INNER JOIN

```
SELECT Table1.Columna, Table2.Columnb

FROM

Table1 INNER JOIN Table2 ON Table1.ColumnID = Table2.ColumnID;
```

ColumnID is the common column between the tables (most likely a foreign key)

The join condition:

Only include **rows** where ColumnID is equal in both tables.

Examples: joins

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

table:Students

```
CREATE TABLE students(
student_id VARCHAR(1),
name CHAR(10),
Course_id VARCHAR(3))

INSERT INTO students (student_id, name, course_id)
VALUES
(1, 'Alice', 101),
(2, 'Bob', 102),
(3, 'Charlie', 103);

SELECT * FROM students
```

table: Courses

```
CREATE TABLE courses(
course_id VARCHAR(3),
course_name CHAR(10))


INSERT INTO courses (course_id, course_name)
VALUES
(101, 'Math'),
(102, 'Sci'),
(104, 'History');

SELECT * FROM courses
```

INNER JOIN

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

INNER JOIN

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

```
SELECT students.name, courses.course_name

FROM  students INNER JOIN courses
ON  students.course_id = courses.course_id;
```

INNER JOIN

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Bob	Science

INNER JOIN

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Bob	Science

Result:
Only Alice and Bob are returned because they have matching course_id values in both tables.

Charlie does not appear because there's no course with course_id 103 in the courses table.

LEFT JOIN (or LEFT OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

LEFT JOIN (or LEFT OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

```
SELECT students.name, courses.course_name
FROM students LEFT JOIN courses
ON students.course_id = courses.course_id;
```

LEFT JOIN (or LEFT OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Bob	Science
Charlie	NULL

LEFT JOIN (or LEFT OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Bob	Science
Charlie	NULL

Result: All students are included, even if they don't have a matching course in the courses table.

Charlie has no matching course, so the course_name is returned as NULL.

RIGHT JOIN (or RIGHT OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

RIGHT JOIN (or RIGHT OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

```
SELECT students.name, courses.course_name

FROM students RIGHT JOIN courses
ON students.course_id = courses.course_id;
```

RIGHT JOIN (or RIGHT OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Bob	Science
NULL	History

RIGHT JOIN (or RIGHT OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Bob	Science
NULL	History

Result: All courses are included, even if there is no matching student.

For the course History, there is no matching student, so the name is NULL.

FULL JOIN (or FULL OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

FULL JOIN (or FULL OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

```
SELECT students.name, courses.course_name
FROM students FULL JOIN courses
ON students.course_id = courses.course_id;
```

FULL JOIN (or FULL OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Bob	Science
Charlie	NULL
NULL	History

FULL JOIN (or FULL OUTER JOIN)

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Bob	Science
Charlie	NULL
NULL	History

Result: This includes all students and all courses.

If a student has no course or a course has no student, NULL values are used for the missing side.

CROSS JOIN

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

```
SELECT students.name, courses.course_name
FROM students CROSS JOIN courses;
```

CROSS JOIN

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Alice	Science
Alice	History
Bob	Math
Bob	Science
Bob	History
Charlie	Math
Charlie	Science
Charlie	History

CROSS JOIN

students table:

student_id	name	course_id
1	Alice	101
2	Bob	102
3	Charlie	103

courses table:

course_id	course_name
101	Math
102	Science
104	History

Result:

name	course_name
Alice	Math
Alice	Science
Alice	History
Bob	Math
Bob	Science
Bob	History
Charlie	Math
Charlie	Science
Charlie	History

Result: Every student is paired with every course. This can produce a lot of rows, so it’s used less frequently than other types of joins.

JOIN + GROUP BY

list all courses along with the number of students enrolled in each course.

JOIN + GROUP BY

list all courses along with the number of students enrolled in each course.

```
SELECT  courses.course_name,
COUNT(students.id)
FROM courses LEFT JOIN students
ON students.course_id1 = courses.course_id2
GROUP BY
Courses.course_name;
```

COUNT(students.id) shows how many students are enrolled in each course.

Courses with no students show 0 because of the LEFT JOIN.

Writing a join query with ON Clause

ON clause

- Used to specify the exact condition to join tables.
- You write a full condition, like table1.columna = table2.columnb.
- Works even if the columns have different names in each table.

Example:

```
SELECT *
FROM basket_a
JOIN basket_b
ON basket_a.fruit_a = basket_b.fruit_b;
```

Writing a join query with USING Clause

USING clause

- A shortcut when joining tables on columns with the same name.
- You only mention the column name once, inside parentheses.
- The result shows the common column only once (no duplicates).

Example:

If both tables have a column named fruit, you can do:

```
SELECT *
FROM table1
JOIN table2
USING (fruit);
```